

GoogLeNet CODE

%Googlenet image NET

close all

clear softmaxLayer % Clear any existing softmaxLayer variable

clear classificationLayer % Clear any existing classificationLayer variable

%unzip('rice.zip');

imds = imageDatastore('Rice_Image_Dataset', 'IncludeSubfolders', true, 'LabelSource', 'foldernames');

[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7, 'randomized');

% Display a random selection of 16 training images

numTrainImages = numel(imdsTrain.Labels);

idx = randperm(numTrainImages, 16);

figure

for i = 1:16

 subplot(4, 4, i)

 I = readimage(imdsTrain, idx(i));

 imshow(I)

end

% Load pre-trained GoogleNet model

net = googlenet;

% Analyze the network structure

analyzeNetwork(net);

% Input size expected by GoogLeNet

inputSize = net.Layers(1).InputSize;

% Prepare the network for transfer learning

lgraph = layerGraph(net); % Create a layer graph for GoogleNet

```

% Remove the last 3 layers (the fully connected and classification
layers)
lgraph = removeLayers(lgraph, {'loss3-classifier', 'prob', 'output'});

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% Add a new fully connected layer and the softmax and
classification layers
new_fc_layer = fullyConnectedLayer(numClasses,
'WeightLearnRateFactor', 20, 'BiasLearnRateFactor', 20, 'Name',
'new_fc');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

% Add these new layers to the graph
lgraph = addLayers(lgraph, new_fc_layer);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the rest of the network
lgraph = connectLayers(lgraph, 'pool5-drop_7x7_s1', 'new_fc'); %
Connect to the last pooling layer
lgraph = connectLayers(lgraph, 'new_fc', 'softmax'); % Connect fully
connected layer to softmax
lgraph = connectLayers(lgraph, 'softmax', 'classoutput'); % Connect
softmax to classification output

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end

```

```

    end
end
fprintf('Total number of parameters: %d\n', totalParams);

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);

% Training options
options = trainingOptions('adam', ...
    'MiniBatchSize', 25, ...
    'MaxEpochs', 15, ...
    'InitialLearnRate', 0.00001, ...
    'LearnRateSchedule', 'none', ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Clear workspace and run garbage collection to get reliable initial
memory
clear temp vars;
[~, sysMem1] = memory;
initialTotalMemory = sysMem1.PhysicalMemory.Total -
sysMem1.PhysicalMemory.Available;
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/
1024/1024);

```

```

% Start measuring training time
tic;

% Train the model
netTransfer = trainNetwork(augimdsTrain, lgraph, options);

% Calculate and display training time
trainingTime = toc;
fprintf('Total training time: %.2f seconds\n', trainingTime);

% Measure memory after training
[~, sysMem2] = memory;
finalTotalMemory = sysMem2.PhysicalMemory.Total -
sysMem2.PhysicalMemory.Available;
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
fprintf('Total memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

% Evaluate the model on the validation set
tic; % Start timing for inference
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc; % Total inference time
YValidation = imdsValidation.Labels;

% Display inference time metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

% Display random predictions and ground truth
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));

```

```

    label2 = YValidation(idx(i));
    % Fixed the label display order
    label = ["Pred -> ", label1, newline, "Actual -> ", label2];
    title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

```

```

% True positives, false positives, false negatives
TP = sum(actualClassIdxs & predictedClassIdxs);
FP = sum(~actualClassIdxs & predictedClassIdxs);
FN = sum(actualClassIdxs & ~predictedClassIdxs);
TN = sum(~actualClassIdxs & ~predictedClassIdxs);

% Class-wise accuracy (binary accuracy for this specific class)
classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;

```

```

    totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics
figure;
uitable('Data', table2cell(macroMetrics), ...
    'ColumnName', macroMetrics.Properties.VariableNames, ...
    'RowName', [], ...

```

```
'Units', 'Normalized', ...  
'Position', [0.2, 0.2, 0.6, 0.6]);  
title('Macro Performance Metrics');  
  
% Save the trained model and results  
save('rice_classification_results.mat', 'netTransfer', 'accuracy',  
'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```


Vgg 19 CODE

%CODE FOR VGG 19

```
close all
clear softmaxLayer % Clear any existing softmaxLayer variable
clear classificationLayer % Clear any existing classificationLayer variable
```

```
%unzip('rice.zip');
imds = imageDatastore('Rice_Image_Dataset_Updated',
'IncludeSubfolders', true, 'LabelSource', 'foldernames');
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7,
'randomized');
```

```
% Display a random selection of 16 training images
numTrainImages = numel(imdsTrain.Labels);
idx = randperm(numTrainImages, 16);
figure
for i = 1:16
    subplot(4, 4, i)
    I = readimage(imdsTrain, idx(i));
    imshow(I)
end
```

```
% Load pre-trained VGG19 model
net = vgg19;
```

```
% Analyze the network structure
analyzeNetwork(net);
```

```
% Input size expected by VGG19
inputSize = net.Layers(1).InputSize;
```

```
% Prepare the network for transfer learning
lgraph = layerGraph(net); % Create a layer graph for VGG19
```

```

% Remove the last 3 layers (the fully connected and classification
layers)
% For VGG19, these layers are different from GoogLeNet
lgraph = removeLayers(lgraph, {'fc8', 'prob', 'output'});

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% Add a new fully connected layer and the softmax and
classification layers
new_fc_layer = fullyConnectedLayer(numClasses,
'WeightLearnRateFactor', 20, 'BiasLearnRateFactor', 20, 'Name',
'new_fc');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

% Add these new layers to the graph
lgraph = addLayers(lgraph, new_fc_layer);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the rest of the network
% In VGG19, the layer before fc8 is 'drop7', not 'fc7'
lgraph = connectLayers(lgraph, 'drop7', 'new_fc');
lgraph = connectLayers(lgraph, 'new_fc', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end

```

```

end
fprintf('Total number of parameters: %d\n', totalParams);

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
    imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
    imdsValidation);

% Training options - slightly modified for VGG19 (lower initial
learning rate)
options = trainingOptions('adam', ...
    'MiniBatchSize', 50, ...          % Batch size of 25 as in your
updated code
    'MaxEpochs', 10, ...            % Fewer epochs since VGG19 is
deeper
    'InitialLearnRate', 0.00001, ... % Learning rate of 0.1 as in
your updated code
    'LearnRateSchedule', 'none', ... % Use piecewise learning rate
schedule
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Clear workspace and run garbage collection to get reliable initial
memory
clear temp vars;
[~, sysMem1] = memory;

```

```
initialTotalMemory = sysMem1.PhysicalMemory.Total -  
sysMem1.PhysicalMemory.Available;  
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/  
1024/1024);
```

```
% Start measuring training time  
tic;
```

```
% Train the model  
netTransfer = trainNetwork(augimdsTrain, lgraph, options);
```

```
% Calculate and display training time  
trainingTime = toc;  
fprintf('Total training time: %.2f seconds\n', trainingTime);
```

```
% Measure memory after training  
[~, sysMem2] = memory;  
finalTotalMemory = sysMem2.PhysicalMemory.Total -  
sysMem2.PhysicalMemory.Available;  
totalMemoryUsed = finalTotalMemory - initialTotalMemory;  
fprintf('Total memory used for training: %.2f MB\n',  
totalMemoryUsed/1024/1024);
```

```
% Evaluate the model on the validation set  
tic; % Start timing for inference  
[YPred, scores] = classify(netTransfer, augimdsValidation);  
inferenceTime = toc; % Total inference time  
YValidation = imdsValidation.Labels;
```

```
% Display inference time metrics  
fprintf('Total inference time: %.2f seconds\n', inferenceTime);  
fprintf('Average inference time per image: %.4f seconds\n',  
inferenceTime/numel(YValidation));
```

```
% Display random predictions and ground truth  
idx = randperm(numel(imdsValidation.Files), 4);  
figure  
for i = 1:4
```

```

subplot(2, 2, i)
I = readimage(imdsValidation, idx(i));
imshow(I)
label1 = YPred(idx(i));
label2 = YValidation(idx(i));
% Fixed the label display order
label = ["Pred -> ", label1, newline, "Actual -> ", label2];
title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

```

```

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

    % True positives, false positives, false negatives
    TP = sum(actualClassIdxs & predictedClassIdxs);
    FP = sum(~actualClassIdxs & predictedClassIdxs);
    FN = sum(actualClassIdxs & ~predictedClassIdxs);
    TN = sum(~actualClassIdxs & ~predictedClassIdxs);

    % Class-wise accuracy (binary accuracy for this specific class)
    classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

    % Precision and recall
    if (TP + FP) > 0
        precision = TP / (TP + FP);
    else
        precision = 0;
    end

    if (TP + FN) > 0
        recall = TP / (TP + FN);
    else
        recall = 0;
    end

    % F1 score
    if (precision + recall) > 0
        f1 = 2 * (precision * recall) / (precision + recall);
    else
        f1 = 0;
    end

    % Store in table
    classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
    classWiseMetrics.Precision(i) = precision * 100;
    classWiseMetrics.Recall(i) = recall * 100;

```

```

    % Add to macro totals
    totalPrecision = totalPrecision + precision;
    totalRecall = totalRecall + recall;
    totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics

```

```
figure;  
uitable('Data', table2cell(macroMetrics), ...  
    'ColumnName', macroMetrics.Properties.VariableNames, ...  
    'RowName', [], ...  
    'Units', 'Normalized', ...  
    'Position', [0.2, 0.2, 0.6, 0.6]);  
title('Macro Performance Metrics');  
  
% Save the trained model and results  
save('rice_classification_vgg19_results.mat', 'netTransfer',  
'accuracy', 'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```


DENSENET201 CODE

%CODE FOR DENSENET-201

```
close all
clear softmaxLayer % Clear any existing softmaxLayer variable
clear classificationLayer % Clear any existing classificationLayer variable

%unzip('rice.zip');
imds = imageDatastore('Rice_Image_Dataset_Updated',
'IncludeSubfolders', true, 'LabelSource', 'foldernames');
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7,
'randomized');

% Display a random selection of 16 training images
numTrainImages = numel(imdsTrain.Labels);
idx = randperm(numTrainImages, 16);
figure
for i = 1:16
    subplot(4, 4, i)
    I = readimage(imdsTrain, idx(i));
    imshow(I)
end

% Load pre-trained DenseNet-201 model
net = densenet201;

% Analyze the network structure
analyzeNetwork(net);

% Input size expected by DenseNet-201
inputSize = net.Layers(1).InputSize;

% Prepare the network for transfer learning
lgraph = layerGraph(net); % Create a layer graph for DenseNet-201
```

```
% Remove the last 3 layers (the fully connected and classification layers)
```

```
% For DenseNet-201, these layers are different from VGG19
```

```
lgraph = removeLayers(lgraph, {'fc1000', 'fc1000_softmax',  
'ClassificationLayer_fc1000'});
```

```
% Number of output classes (categories of rice types)
```

```
numClasses = numel(categories(imdsTrain.Labels));
```

```
% Add a new fully connected layer and the softmax and classification layers
```

```
new_fc_layer = fullyConnectedLayer(numClasses,  
'WeightLearnRateFactor', 20, 'BiasLearnRateFactor', 20, 'Name',  
'new_fc');
```

```
smLayer = softmaxLayer('Name', 'softmax');
```

```
classLayer = classificationLayer('Name', 'classoutput');
```

```
% Add these new layers to the graph
```

```
lgraph = addLayers(lgraph, new_fc_layer);
```

```
lgraph = addLayers(lgraph, smLayer);
```

```
lgraph = addLayers(lgraph, classLayer);
```

```
% Connect the new layers to the rest of the network
```

```
% In DenseNet-201, the layer before fc1000 is 'avg_pool'
```

```
lgraph = connectLayers(lgraph, 'avg_pool', 'new_fc');
```

```
lgraph = connectLayers(lgraph, 'new_fc', 'softmax');
```

```
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');
```

```
% Calculate total number of parameters in the network
```

```
totalParams = 0;
```

```
for i = 1:numel(lgraph.Layers)
```

```
    if isprop(lgraph.Layers(i), 'Weights') &&
```

```
    ~isempty(lgraph.Layers(i).Weights)
```

```
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
```

```
    end
```

```
    if isprop(lgraph.Layers(i), 'Bias') &&
```

```
    ~isempty(lgraph.Layers(i).Bias)
```

```
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
```

```

    end
end
fprintf('Total number of parameters: %d\n', totalParams);

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);

% Training options - adjusted for DenseNet-201
options = trainingOptions('adam', ...
    'MiniBatchSize', 12, ...          % Adjusted batch size for
DenseNet-201
    'MaxEpochs', 10, ...
    'InitialLearnRate', 0.00001, ... % Keep the small learning rate
    'LearnRateSchedule', 'none', ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Clear workspace and run garbage collection to get reliable initial
memory
clear temp vars;
[~, sysMem1] = memory;
initialTotalMemory = sysMem1.PhysicalMemory.Total -
sysMem1.PhysicalMemory.Available;

```

```
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/  
1024/1024);
```

```
% Start measuring training time  
tic;
```

```
% Train the model  
netTransfer = trainNetwork(augimdsTrain, lgraph, options);
```

```
% Calculate and display training time  
trainingTime = toc;  
fprintf('Total training time: %.2f seconds\n', trainingTime);
```

```
% Measure memory after training  
[~, sysMem2] = memory;  
finalTotalMemory = sysMem2.PhysicalMemory.Total -  
sysMem2.PhysicalMemory.Available;  
totalMemoryUsed = finalTotalMemory - initialTotalMemory;  
fprintf('Total memory used for training: %.2f MB\n',  
totalMemoryUsed/1024/1024);
```

```
% Evaluate the model on the validation set  
tic; % Start timing for inference  
[YPred, scores] = classify(netTransfer, augimdsValidation);  
inferenceTime = toc; % Total inference time  
YValidation = imdsValidation.Labels;
```

```
% Display inference time metrics  
fprintf('Total inference time: %.2f seconds\n', inferenceTime);  
fprintf('Average inference time per image: %.4f seconds\n',  
inferenceTime/numel(YValidation));
```

```
% Display random predictions and ground truth  
idx = randperm(numel(imdsValidation.Files), 4);  
figure  
for i = 1:4  
    subplot(2, 2, i)  
    I = readimage(imdsValidation, idx(i));
```

```

imshow(I)
label1 = YPred(idx(i));
label2 = YValidation(idx(i));
% Fixed the label display order
label = ["Pred -> ", label1, newline, "Actual -> ", label2];
title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class

```

```

actualClassIdxs = YValidation == classes{i};
predictedClassIdxs = YPred == classes{i};

% True positives, false positives, false negatives
TP = sum(actualClassIdxs & predictedClassIdxs);
FP = sum(~actualClassIdxs & predictedClassIdxs);
FN = sum(actualClassIdxs & ~predictedClassIdxs);
TN = sum(~actualClassIdxs & ~predictedClassIdxs);

% Class-wise accuracy (binary accuracy for this specific class)
classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals

```

```

    totalPrecision = totalPrecision + precision;
    totalRecall = totalRecall + recall;
    totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics
figure;
uitable('Data', table2cell(macroMetrics), ...

```

```
'ColumnName', macroMetrics.Properties.VariableNames, ...  
'RowName', [], ...  
'Units', 'Normalized', ...  
'Position', [0.2, 0.2, 0.6, 0.6]);  
title('Macro Performance Metrics');  
  
% Save the trained model and results  
save('rice_classification_densenet201_results.mat', 'netTransfer',  
'accuracy', 'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```


AlexNet CODE

```
%Alexnet
```

```
close all
```

```
clear softmaxLayer % Clear any existing softmaxLayer variable
```

```
clear classificationLayer % Clear any existing classificationLayer variable
```

```
%unzip('rice.zip');
```

```
imds = imageDatastore('Rice_Image_Dataset_Updated',  
'IncludeSubfolders', true, 'LabelSource', 'foldernames');
```

```
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7,  
'randomized');
```

```
% Display a random selection of 16 training images
```

```
numTrainImages = numel(imdsTrain.Labels);
```

```
idx = randperm(numTrainImages, 16);
```

```
figure
```

```
for i = 1:16
```

```
    subplot(4, 4, i)
```

```
    I = readimage(imdsTrain, idx(i));
```

```
    imshow(I)
```

```
end
```

```
% Load pre-trained AlexNet model
```

```
net = alexnet;
```

```
% Analyze the network structure
```

```
analyzeNetwork(net);
```

```
% Input size expected by AlexNet (227x227 instead of GoogLeNet's  
224x224)
```

```
inputSize = net.Layers(1).InputSize;
```

```
% Prepare the network for transfer learning
```

```
lgraph = layerGraph(net); % Create a layer graph for AlexNet
```

```

% Remove the last 3 layers (the fully connected and classification
layers)
lgraph = removeLayers(lgraph, {'fc8', 'prob', 'output'});

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% Add a new fully connected layer and the softmax and
classification layers
new_fc_layer = fullyConnectedLayer(numClasses,
'WeightLearnRateFactor', 20, 'BiasLearnRateFactor', 20, 'Name',
'new_fc');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

% Add these new layers to the graph
lgraph = addLayers(lgraph, new_fc_layer);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the rest of the network
lgraph = connectLayers(lgraph, 'drop7', 'new_fc'); % Connect to the
last dropout layer in AlexNet
lgraph = connectLayers(lgraph, 'new_fc', 'softmax'); % Connect fully
connected layer to softmax
lgraph = connectLayers(lgraph, 'softmax', 'classoutput'); % Connect
softmax to classification output

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end

```

```

    end
end
fprintf('Total number of parameters: %d\n', totalParams);

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);

% Training options
options = trainingOptions('adam', ...
    'MiniBatchSize', 25, ...
    'MaxEpochs', 10, ...
    'InitialLearnRate', 0.00001, ...
    'LearnRateSchedule', 'none', ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Clear workspace and run garbage collection to get reliable initial
memory
clear temp vars;
[~, sysMem1] = memory;
initialTotalMemory = sysMem1.PhysicalMemory.Total -
sysMem1.PhysicalMemory.Available;
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/
1024/1024);

```

```

% Start measuring training time
tic;

% Train the model
netTransfer = trainNetwork(augimdsTrain, lgraph, options);

% Calculate and display training time
trainingTime = toc;
fprintf('Total training time: %.2f seconds\n', trainingTime);

% Measure memory after training
[~, sysMem2] = memory;
finalTotalMemory = sysMem2.PhysicalMemory.Total -
sysMem2.PhysicalMemory.Available;
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
fprintf('Total memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

% Evaluate the model on the validation set
tic; % Start timing for inference
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc; % Total inference time
YValidation = imdsValidation.Labels;

% Display inference time metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

% Display random predictions and ground truth
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));

```

```

    label2 = YValidation(idx(i));
    % Fixed the label display order
    label = ["Pred -> ", label1, newline, "Actual -> ", label2];
    title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

```

```

% True positives, false positives, false negatives
TP = sum(actualClassIdxs & predictedClassIdxs);
FP = sum(~actualClassIdxs & predictedClassIdxs);
FN = sum(actualClassIdxs & ~predictedClassIdxs);
TN = sum(~actualClassIdxs & ~predictedClassIdxs);

% Class-wise accuracy (binary accuracy for this specific class)
classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;

```

```

    totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics
figure;
uitable('Data', table2cell(macroMetrics), ...
    'ColumnName', macroMetrics.Properties.VariableNames, ...
    'RowName', [], ...

```

```
'Units', 'Normalized', ...  
'Position', [0.2, 0.2, 0.6, 0.6]);  
title('Macro Performance Metrics');  
  
% Save the trained model and results  
save('rice_classification_alexnet_results.mat', 'netTransfer',  
'accuracy', 'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```


GoogLeNet places 365 CODE

%GOOGLNET PLACES 365

%% Rice Classification using Transfer Learning with GoogLeNet

% This script implements transfer learning using GoogLeNet for rice classification

% Clear workspace and close figures

close all

clear all

%% Load and prepare data

% Load image data

imds = imageDatastore('Rice_Image_Dataset_Updated',
'IncludeSubfolders', true, 'LabelSource', 'foldernames');

% Split data into training and validation sets (70/30 split)

[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7,
'randomized');

% Display sample training images

numTrainImages = numel(imdsTrain.Labels);

idx = randperm(numTrainImages, 16);

figure

for i = 1:16

 subplot(4, 4, i)

 I = readimage(imdsTrain, idx(i));

 imshow(I)

end

sgtitle('Sample Training Images', 'FontSize', 14);

%% Load and modify pre-trained network

% Load pre-trained GoogleNet model

[net, ~] = imagePretrainedNetwork("googlenet-places365");

% Input size expected by GoogLeNet

inputSize = net.Layers(1).InputSize;

```

% Prepare the network for transfer learning
lgraph = layerGraph(net);

% Analyze the network to find the correct layers
layerNames = {net.Layers.Name};
fprintf('Network has %d layers\n', length(layerNames));

% Find the last few layers
finalLayers = layerNames(end-2:end);
fprintf('Final three layers: %s, %s, %s\n', finalLayers{1},
finalLayers{2}, finalLayers{3});

% Find a suitable layer to connect to by identifying the drop layer or
avgpool layer
% Common feature layers in GoogLeNets have names containing
'drop', 'pool', or 'relu'
featureLayerIdx = [];
for i = length(layerNames)-3:-1:1 % Search backward from the end
(excluding last 3 layers)
    currentName = layerNames{i};
    if contains(lower(currentName), 'drop') ||
contains(lower(currentName), 'pool')
        featureLayerIdx = i;
        break;
    end
end

% If no suitable layer found, try layers with 'relu'
if isempty(featureLayerIdx)
    for i = length(layerNames)-3:-1:1
        if contains(lower(layerNames{i}), 'relu')
            featureLayerIdx = i;
            break;
        end
    end
end
end

```

```

% If still no suitable layer, use the layer right before the last 3
if isempty(featureLayerIdx)
    featureLayerIdx = length(layerNames) - 3;
end

featureLayer = layerNames{featureLayerIdx};
fprintf('Using layer %d ("%s") as feature extraction layer\n',
featureLayerIdx, featureLayer);

% Remove the last 3 layers (fully connected and classification
layers)
lgraph = removeLayers(lgraph, finalLayers);

% Get number of classes from the dataset
numClasses = numel(categories(imdsTrain.Labels));
fprintf('Number of classes: %d\n', numClasses);

% Add new classification layers with higher learn rates for faster
adaptation
new_fc_layer = fullyConnectedLayer(numClasses,
'WeightLearnRateFactor', 20, 'BiasLearnRateFactor', 20, 'Name',
'new_fc');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

% Add these new layers to the network
lgraph = addLayers(lgraph, new_fc_layer);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the network
lgraph = connectLayers(lgraph, featureLayer, 'new_fc');
lgraph = connectLayers(lgraph, 'new_fc', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');

% Plot the network architecture (optional - uncomment if needed)
% figure;
% plot(lgraph);

```

```

% title('Modified GoogLeNet Architecture');

% Calculate network complexity (total parameters)
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end
fprintf('Total number of parameters: %d\n', totalParams);

%% Data Augmentation
% Create augmenter for training data
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange, ...
    'RandRotation', [-15, 15]); % Added rotation for better
generalization

% Create augmented image datastores
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);

%% Training Options
% Set up training options
options = trainingOptions('rmsprop', ...
    'MiniBatchSize', 25, ... % Reduced batch size to help with
memory usage

```

```
'MaxEpochs', 10, ...  
'InitialLearnRate', 0.00001, ... % Increased slightly for faster  
learning  
'LearnRateSchedule', 'none', ... % Added learning rate decay  
'Shuffle', 'every-epoch', ...  
'ValidationData', augimdsValidation, ...  
'ValidationFrequency', 1, ... % Validate more frequently  
'Verbose', false, ...  
'Plots', 'training-progress', ...  
'ExecutionEnvironment', 'auto'); % Let MATLAB decide optimal  
execution environment
```

```
%% Memory and Performance Monitoring
```

```
% Clear temporary variables and measure initial memory
```

```
clear temp vars;
```

```
[~, sysMem1] = memory;
```

```
initialTotalMemory = sysMem1.PhysicalMemory.Total -
```

```
sysMem1.PhysicalMemory.Available;
```

```
fprintf('Initial memory usage: %.2f MB\n', initialTotalMemory/  
1024/1024);
```

```
%% Train Network
```

```
% Start measuring training time
```

```
tic;
```

```
% Train the model
```

```
netTransfer = trainNetwork(augimdsTrain, lgraph, options);
```

```
% Calculate and display training time
```

```
trainingTime = toc;
```

```
fprintf('Total training time: %.2f seconds\n', trainingTime);
```

```
% Measure memory after training
```

```
[~, sysMem2] = memory;
```

```
finalTotalMemory = sysMem2.PhysicalMemory.Total -
```

```
sysMem2.PhysicalMemory.Available;
```

```
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
```

```

fprintf('Additional memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

%% Evaluate Model
% Measure inference time
tic;
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc;
YValidation = imdsValidation.Labels;

% Display inference metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

%% Display Results
% Show sample predictions
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));
    label2 = YValidation(idx(i));
    title(sprintf('Pred: %s\nActual: %s', string(label1), string(label2)));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

%% Create Confusion Matrix
% Get class categories
classes = categories(YValidation);

% Create and display confusion matrix
figure;

```

```
cm = confusionmat(YValidation, YPred);
confChart = confusionchart(cm, classes, 'Normalization', 'row-
normalized', 'Title', 'Confusion Matrix');
confChart.ColumnSummary = 'column-normalized';
confChart.RowSummary = 'row-normalized';
```

```
%% Detailed Performance Metrics
```

```
% Create table for class-wise metrics
```

```
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Precision', 'Recall', 'F1Score'});
```

```
classWiseMetrics.Class = classes;
```

```
% Initialize variables for macro metrics
```

```
totalPrecision = 0;
```

```
totalRecall = 0;
```

```
totalF1 = 0;
```

```
classAccuracies = zeros(length(classes), 1);
```

```
% Calculate metrics for each class
```

```
for i = 1:numel(classes)
```

```
    % Get indices for current class
```

```
    actualClassIdxs = YValidation == classes{i};
```

```
    predictedClassIdxs = YPred == classes{i};
```

```
    % Calculate metrics
```

```
    TP = sum(actualClassIdxs & predictedClassIdxs);
```

```
    FP = sum(~actualClassIdxs & predictedClassIdxs);
```

```
    FN = sum(actualClassIdxs & ~predictedClassIdxs);
```

```
    TN = sum(~actualClassIdxs & ~predictedClassIdxs);
```

```
    % Class-wise binary accuracy
```

```
    classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);
```

```
    % Precision
```

```
    if (TP + FP) > 0
```

```
        precision = TP / (TP + FP);
```

```
else
    precision = 0;
end
```

```
% Recall
if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end
```

```
% F1 score - calculate directly to avoid division by zero issues
if precision + recall > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end
```

```
% Store in table
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;
classWiseMetrics.F1Score(i) = f1 * 100;
```

```
% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;
totalF1 = totalF1 + f1;
```

```
end
```

```
% Calculate macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = 2 * (macroPrecision * macroRecall) / (macroPrecision +
macroRecall + eps); % Added eps to avoid div by zero
macroAccuracy = mean(classAccuracies);
```

```
% Display class-wise metrics
disp('Class-wise Performance Metrics (%):');
```



```

disp(classWiseMetrics);

% Display macro metrics
fprintf('\nMacro-averaged Metrics:\n');
fprintf('Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Precision: %.2f%%\n', macroPrecision*100);
fprintf('Recall: %.2f%%\n', macroRecall*100);
fprintf('F1 Score: %.2f%%\n', macroF1*100);

%% Create Summary Visualization
% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
    macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary table
figure;
uitable('Data', table2cell(macroMetrics), ...
    'ColumnName', macroMetrics.Properties.VariableNames, ...
    'RowName', [], ...
    'Units', 'Normalized', ...
    'Position', [0.2, 0.2, 0.6, 0.6]);
title('Macro Performance Metrics', 'FontSize', 14);

% Create bar chart for class-wise F1 scores
figure;
bar(classWiseMetrics.F1Score);
xlabel('Class', 'FontSize', 12);
ylabel('F1 Score (%)', 'FontSize', 12);
title('F1 Score by Class', 'FontSize', 14);
xticks(1:numel(classes));
xticklabels(classes);
xtickangle(45);
grid on;

%% Save Results
% Save the trained model and results

```

```
save('rice_classification_results.mat', 'netTransfer', 'accuracy',  
'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```

```
% Display completion message  
fprintf('\nTraining and evaluation complete. Results saved to  
rice_classification_results.mat\n');
```

NasNet Large CODE

%CODE FOR NASNetLarge

```
close all
clear softmaxLayer % Clear any existing softmaxLayer variable
clear classificationLayer % Clear any existing classificationLayer variable
```

```
%unzip('rice.zip');
imds = imageDatastore('Rice_Image_Dataset_Updated',
'IncludeSubfolders', true, 'LabelSource', 'foldernames');
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.5,
'randomized');
```

```
% Display a random selection of 16 training images
numTrainImages = numel(imdsTrain.Labels);
idx = randperm(numTrainImages, 16);
figure
for i = 1:16
    subplot(4, 4, i)
    I = readimage(imdsTrain, idx(i));
    imshow(I)
end
```

```
% Load pre-trained NASNetLarge model
net = nasnetlarge;
```

```
% Analyze the network structure
analyzeNetwork(net);
```

```
% Input size expected by NASNetLarge
inputSize = net.Layers(1).InputSize;
```

```
% Prepare the network for transfer learning
lgraph = layerGraph(net);
```

```
% Print out the last several layers to understand the structure
```

```

fprintf('Last layers in the network:\n');
numLayers = length(lgraph.Layers);
numToShow = min(10, numLayers);
layerNames = {lgraph.Layers(max(1, numLayers-
numToShow+1):numLayers).Name};
for i = 1:length(layerNames)
    fprintf(' - %s\n', layerNames{i});
end

% Remove the classification layers
layersToRemove = {'predictions', 'predictions_softmax',
'ClassificationLayer_predictions'};
existingLayers = {lgraph.Layers.Name};
layersToRemoveFiltered =
layersToRemove(ismember(layersToRemove, existingLayers));

fprintf('Removing layers: %s\n', strjoin(layersToRemoveFiltered, '
'));
lgraph = removeLayers(lgraph, layersToRemoveFiltered);

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% Add new layers for classification
new_fc_layer = fullyConnectedLayer(numClasses,
'WeightLearnRateFactor', 20, 'BiasLearnRateFactor', 20, 'Name',
'new_fc');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

% Add these new layers to the graph
lgraph = addLayers(lgraph, new_fc_layer);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Find the appropriate pooling layer to connect from
poolLayerName = '';
for i = 1:numel(lgraph.Layers)

```

```

    if contains(lower(class(lgraph.Layers(i))), 'globalaveragepool')
        poolLayerName = lgraph.Layers(i).Name;
        break;
    end
end

% If not found by class, try common names
if isempty(poolLayerName)
    commonPoolNames = {'global_average_pooling2d_2',
'global_average_pooling2d_1', 'avg_pool', 'averagepool',
'global_avg_pool'};
    for name = commonPoolNames
        if any(strcmp(existingLayers, name{1}))
            poolLayerName = name{1};
            break;
        end
    end
end

% If still not found, look for terminal layers
if isempty(poolLayerName)
    terminalLayers = {};
    for i = 1:numel(existingLayers)
        layerName = existingLayers{i};
        if isempty(lgraph.OutputNames(layerName))
            terminalLayers{end+1} = layerName;
        end
    end

    if ~isempty(terminalLayers)
        if length(terminalLayers) == 1
            poolLayerName = terminalLayers{1};
        else
            % Try to find a terminal layer that looks like a pooling layer
            for i = 1:length(terminalLayers)
                if contains(lower(terminalLayers{i}), {'pool', 'avg',
'average', 'global'})
                    poolLayerName = terminalLayers{i};
                end
            end
        end
    end
end

```

```

        break;
    end
end

    % If still not found, use the first terminal layer
    if isempty(poolLayerName) && ~isempty(terminalLayers)
        poolLayerName = terminalLayers{1};
    end
end
end
end

fprintf('Connecting from layer: %s\n', poolLayerName);
lgraph = connectLayers(lgraph, poolLayerName, 'new_fc');
lgraph = connectLayers(lgraph, 'new_fc', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end
end
fprintf('Total number of parameters: %d\n', totalParams);

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

```

```
% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
```

```
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);
```

```
% Training options for NASNetLarge
```

```
options = trainingOptions('adam', ...
    'MiniBatchSize', 50, ... % Smaller batch size for
NASNetLarge
    'MaxEpochs', 1, ...
    'InitialLearnRate', 0.00001, ... % Lower learning rate for
NASNetLarge
    'LearnRateSchedule', 'none', ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
```

```
% Clear workspace and run garbage collection to get reliable initial
memory
```

```
clear temp vars;
[~, sysMem1] = memory;
initialTotalMemory = sysMem1.PhysicalMemory.Total -
sysMem1.PhysicalMemory.Available;
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/
1024/1024);
```

```
% Start measuring training time
tic;
```

```
% Train the model
```

```
fprintf('Starting training...\n');
netTransfer = trainNetwork(augimdsTrain, lgraph, options);
```

```
% Calculate and display training time
```

```

trainingTime = toc;
fprintf('Total training time: %.2f seconds\n', trainingTime);

% Measure memory after training
[~, sysMem2] = memory;
finalTotalMemory = sysMem2.PhysicalMemory.Total -
sysMem2.PhysicalMemory.Available;
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
fprintf('Total memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

% Evaluate the model on the validation set
tic; % Start timing for inference
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc; % Total inference time
YValidation = imdsValidation.Labels;

% Display inference time metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

% Display random predictions and ground truth
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));
    label2 = YValidation(idx(i));
    label = ["Pred -> ", label1, newline, "Actual -> ", label2];
    title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

```



```

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

    % True positives, false positives, false negatives
    TP = sum(actualClassIdxs & predictedClassIdxs);
    FP = sum(~actualClassIdxs & predictedClassIdxs);
    FN = sum(actualClassIdxs & ~predictedClassIdxs);
    TN = sum(~actualClassIdxs & ~predictedClassIdxs);

    % Class-wise accuracy (binary accuracy for this specific class)
    classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

```

```

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;
totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

```

```

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics
figure;
uitable('Data', table2cell(macroMetrics), ...
    'ColumnName', macroMetrics.Properties.VariableNames, ...
    'RowName', [], ...
    'Units', 'Normalized', ...
    'Position', [0.2, 0.2, 0.6, 0.6]);
title('Macro Performance Metrics');

% Save the trained model and results
save('rice_classification_nasnetlarge_results.mat', 'netTransfer',
'accuracy', 'trainingTime', ...
    'totalParams', 'totalMemoryUsed', 'inferenceTime',
'classWiseMetrics', 'macroMetrics');

```

%CODE FOR DROPOUTS

%ALEXNET

%Alexnet with Custom Dropout

%AlexNet Transfer Learning for Rice Classification with Custom Dropout

close all

clear softmaxLayer % Clear any existing softmaxLayer variable

clear classificationLayer % Clear any existing classificationLayer variable

%unzip('rice.zip');

imds = imageDatastore('Rice_Image_Dataset_Updated',

'IncludeSubfolders', true, 'LabelSource', 'foldernames');

[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7, 'randomized');

% Display a random selection of 16 training images

numTrainImages = numel(imdsTrain.Labels);

idx = randperm(numTrainImages, 16);

figure

for i = 1:16

 subplot(4, 4, i)

 I = readimage(imdsTrain, idx(i));

 imshow(I)

end

% Load pre-trained AlexNet model

net = alexnet;

% Analyze the network structure

analyzeNetwork(net);

% Input size expected by AlexNet (227x227 instead of GoogLeNet's 224x224)

```

inputSize = net.Layers(1).InputSize;

% Prepare the network for transfer learning
lgraph = layerGraph(net); % Create a layer graph for AlexNet

% Remove the last 3 layers (the fully connected and classification
layers)
lgraph = removeLayers(lgraph, {'fc8', 'prob', 'output'});

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% ===== CUSTOM DROPOUT MODIFICATION
=====
% Create a new fully connected layer with adjusted size
intermediateLayerSize = 256; % Size of the intermediate layer
intermediate_fc_layer =
fullyConnectedLayer(intermediateLayerSize, ...
    'WeightLearnRateFactor', 10, ...
    'BiasLearnRateFactor', 10, ...
    'Name', 'fc_intermediate');

% Create a custom dropout layer with 50% dropout rate
% Higher dropout rate (0.5) for better regularization
customDropoutLayer = dropoutLayer(0.75, 'Name',
'custom_dropout');

% Create the final classification layers
final_fc_layer = fullyConnectedLayer(numClasses, ...
    'WeightLearnRateFactor', 20, ...
    'BiasLearnRateFactor', 20, ...
    'Name', 'fc_final');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

% Add these new layers to the graph
lgraph = addLayers(lgraph, intermediate_fc_layer);
lgraph = addLayers(lgraph, customDropoutLayer);

```

```

lgraph = addLayers(lgraph, final_fc_layer);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the rest of the network
lgraph = connectLayers(lgraph, 'drop7', 'fc_intermediate');
lgraph = connectLayers(lgraph, 'fc_intermediate', 'custom_dropout');
lgraph = connectLayers(lgraph, 'custom_dropout', 'fc_final');
lgraph = connectLayers(lgraph, 'fc_final', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');
% ===== END OF CUSTOM DROPOUT MODIFICATION
=====

% Display the modified network
figure
plot(lgraph)
title('Modified AlexNet with Custom Dropout')

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end
fprintf('Total number of parameters: %d\n', totalParams);

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

```

```
% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
```

```
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);
```

```
% Training options with learning rate drop parameters
```

```
options = trainingOptions('adam', ...
```

```
    'MiniBatchSize', 12, ...
```

```
    'MaxEpochs', 10, ... % Increased epochs to better see dropout
effects
```

```
    'InitialLearnRate', 0.00001, ... % Increased learning rate
```

```
    'LearnRateSchedule', 'NONE', ...
```

```
    'Shuffle', 'every-epoch', ...
```

```
    'ValidationData', augimdsValidation, ...
```

```
    'ValidationFrequency', 1, ...
```

```
    'Verbose', false, ...
```

```
    'Plots', 'training-progress');
```

```
% Clear workspace and run garbage collection to get reliable initial
memory
```

```
clear temp vars;
```

```
[~, sysMem1] = memory;
```

```
initialTotalMemory = sysMem1.PhysicalMemory.Total -
```

```
sysMem1.PhysicalMemory.Available;
```

```
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/
1024/1024);
```

```
% Start measuring training time
```

```
tic;
```

```
% Train the model
```

```
netTransfer = trainNetwork(augimdsTrain, lgraph, options);
```

```
% Calculate and display training time
```

```
trainingTime = toc;
```

```

fprintf('Total training time: %.2f seconds\n', trainingTime);

% Measure memory after training
[~, sysMem2] = memory;
finalTotalMemory = sysMem2.PhysicalMemory.Total -
sysMem2.PhysicalMemory.Available;
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
fprintf('Total memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

% Evaluate the model on the validation set
tic; % Start timing for inference
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc; % Total inference time
YValidation = imdsValidation.Labels;

% Display inference time metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

% Display random predictions and ground truth
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));
    label2 = YValidation(idx(i));
    % Fixed the label display order
    label = ["Pred -> ", label1, newline, "Actual -> ", label2];
    title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

```



```

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

    % True positives, false positives, false negatives
    TP = sum(actualClassIdxs & predictedClassIdxs);
    FP = sum(~actualClassIdxs & predictedClassIdxs);
    FN = sum(actualClassIdxs & ~predictedClassIdxs);
    TN = sum(~actualClassIdxs & ~predictedClassIdxs);

    % Class-wise accuracy (binary accuracy for this specific class)
    classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

```

```

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;
totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

```

```

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics
figure;
uitable('Data', table2cell(macroMetrics), ...
    'ColumnName', macroMetrics.Properties.VariableNames, ...
    'RowName', [], ...
    'Units', 'Normalized', ...
    'Position', [0.2, 0.2, 0.6, 0.6]);
title('Macro Performance Metrics');

% Save the trained model and results
save('rice_classification_alexnet_dropout_results.mat',
'netTransfer', 'accuracy', 'trainingTime', ...
    'totalParams', 'totalMemoryUsed', 'inferenceTime',
'classWiseMetrics', 'macroMetrics');

```

%Code for DENSENET 201

%CODE FOR DENSENET201 WITH DROPOUT

%CODE FOR DENSENET-201 WITH CUSTOM DROPOUT
LAYERS

close all

clear softmaxLayer % Clear any existing softmaxLayer variable

clear classificationLayer % Clear any existing classificationLayer
variable

%unzip('rice.zip');

imds = imageDatastore('Rice_Image_Dataset_Updated',
'IncludeSubfolders', true, 'LabelSource', 'foldernames');
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7,
'randomized');

% Display a random selection of 16 training images

numTrainImages = numel(imdsTrain.Labels);

idx = randperm(numTrainImages, 16);

figure

for i = 1:16

 subplot(4, 4, i)

 I = readimage(imdsTrain, idx(i));

 imshow(I)

end

% Load pre-trained DenseNet-201 model

net = densenet201;

% Analyze the network structure

analyzeNetwork(net);

% Input size expected by DenseNet-201

inputSize = net.Layers(1).InputSize;

% Prepare the network for transfer learning

```

lgraph = layerGraph(net); % Create a layer graph for DenseNet-201

% Remove the last 3 layers (the fully connected and classification
layers)
lgraph = removeLayers(lgraph, {'fc1000', 'fc1000_softmax',
'ClassificationLayer_fc1000'});

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% ===== CUSTOM DROPOUT IMPLEMENTATION
=====
% Create intermediate fully connected layers with dropout
intermediateSize1 = 512; % First intermediate layer size
intermediateSize2 = 256; % Second intermediate layer size

% First intermediate layer with dropout
intermediate_fc1 = fullyConnectedLayer(intermediateSize1, ...
    'WeightLearnRateFactor', 10, ...
    'BiasLearnRateFactor', 10, ...
    'Name', 'fc_intermediate1');
dropout1 = dropoutLayer(0.95, 'Name', 'dropout1'); % 40% dropout

% Second intermediate layer with dropout
intermediate_fc2 = fullyConnectedLayer(intermediateSize2, ...
    'WeightLearnRateFactor', 15, ...
    'BiasLearnRateFactor', 15, ...
    'Name', 'fc_intermediate2');
dropout2 = dropoutLayer(0.95, 'Name', 'dropout2'); % 50% dropout

% Final output layer
final_fc = fullyConnectedLayer(numClasses, ...
    'WeightLearnRateFactor', 20, ...
    'BiasLearnRateFactor', 20, ...
    'Name', 'fc_final');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

```

```

% Add these new layers to the graph
lgraph = addLayers(lgraph, intermediate_fc1);
lgraph = addLayers(lgraph, dropout1);
lgraph = addLayers(lgraph, intermediate_fc2);
lgraph = addLayers(lgraph, dropout2);
lgraph = addLayers(lgraph, final_fc);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the rest of the network
lgraph = connectLayers(lgraph, 'avg_pool', 'fc_intermediate1');
lgraph = connectLayers(lgraph, 'fc_intermediate1', 'dropout1');
lgraph = connectLayers(lgraph, 'dropout1', 'fc_intermediate2');
lgraph = connectLayers(lgraph, 'fc_intermediate2', 'dropout2');
lgraph = connectLayers(lgraph, 'dropout2', 'fc_final');
lgraph = connectLayers(lgraph, 'fc_final', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');
% ===== END OF CUSTOM DROPOUT IMPLEMENTATION
=====

% Display the modified network
figure
plot(lgraph)
title('Modified DenseNet-201 with Custom Dropout Layers')

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end
fprintf('Total number of parameters: %d\n', totalParams);

```

```

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange, ...
    'RandRotation', [-15, 15]); % Adding rotation augmentation

% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);

% Training options - adjusted for DenseNet-201 with dropout
options = trainingOptions('adam', ...
    'MiniBatchSize', 25, ...
    'MaxEpochs', 10, ... % Increased epochs for better
training
    'InitialLearnRate', 0.00001, ... % Increased learning rate
    'LearnRateSchedule', 'none', ... % Every 5 epochs
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ... % Early stopping if no
improvement for 5 epochs
    'Verbose', false, ...
    'Plots', 'training-progress');

% Clear workspace and run garbage collection to get reliable initial
memory
clear temp vars;
[~, sysMem1] = memory;
initialTotalMemory = sysMem1.PhysicalMemory.Total -
sysMem1.PhysicalMemory.Available;
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/
1024/1024);

```

```

% Start measuring training time
tic;

% Train the model
netTransfer = trainNetwork(augimdsTrain, lgraph, options);

% Calculate and display training time
trainingTime = toc;
fprintf('Total training time: %.2f seconds\n', trainingTime);

% Measure memory after training
[~, sysMem2] = memory;
finalTotalMemory = sysMem2.PhysicalMemory.Total -
sysMem2.PhysicalMemory.Available;
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
fprintf('Total memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

% Evaluate the model on the validation set
tic; % Start timing for inference
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc; % Total inference time
YValidation = imdsValidation.Labels;

% Display inference time metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

% Display random predictions and ground truth
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));

```



```

    label2 = YValidation(idx(i));
    % Fixed the label display order
    label = ["Pred -> ", label1, newline, "Actual -> ", label2];
    title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

```

```

% True positives, false positives, false negatives
TP = sum(actualClassIdxs & predictedClassIdxs);
FP = sum(~actualClassIdxs & predictedClassIdxs);
FN = sum(actualClassIdxs & ~predictedClassIdxs);
TN = sum(~actualClassIdxs & ~predictedClassIdxs);

% Class-wise accuracy (binary accuracy for this specific class)
classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;

```

```

    totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics
figure;
uitable('Data', table2cell(macroMetrics), ...
    'ColumnName', macroMetrics.Properties.VariableNames, ...
    'RowName', [], ...

```

```
'Units', 'Normalized', ...  
'Position', [0.2, 0.2, 0.6, 0.6]);  
title('Macro Performance Metrics');  
  
% Save the trained model and results  
save('rice_classification_densenet201_dropout_results.mat',  
'netTransfer', 'accuracy', 'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```

```
%CODE FOT GOOGLE NET
```

```
%Googlenet with Dropout Only
```

```
close all
```

```
clear softmaxLayer % Clear any existing softmaxLayer variable
```

```
clear classificationLayer % Clear any existing classificationLayer  
variable
```

```
%unzip('rice.zip');
```

```
imds = imageDatastore('Rice_Image_Dataset_Updated',  
'IncludeSubfolders', true, 'LabelSource', 'foldernames');  
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7,  
'randomized');
```

```
% Display a random selection of 16 training images
```

```
numTrainImages = numel(imdsTrain.Labels);
```

```
idx = randperm(numTrainImages, 16);
```

```
figure
```

```
for i = 1:16
```

```
    subplot(4, 4, i)
```

```
    I = readimage(imdsTrain, idx(i));
```

```
    imshow(I)
```

```
end
```

```
% Load pre-trained GoogleNet model
```

```
net = googlenet;
```

```
% Analyze the network structure
```

```
analyzeNetwork(net);
```

```
% Input size expected by GoogLeNet
```

```
inputSize = net.Layers(1).InputSize;
```

```
% Prepare the network for transfer learning
```

```
lgraph = layerGraph(net); % Create a layer graph for GoogleNet
```

```

% Remove the last 3 layers (the fully connected and classification
layers)
lgraph = removeLayers(lgraph, {'loss3-classifier', 'prob', 'output'});

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% ===== CUSTOM DROPOUT ARCHITECTURE
=====
% Define new layers with dropout
% Note: pool5-drop_7x7_s1 already has dropout, but we'll add
more

% Create intermediate layers with dropout
intermediate_fc1 = fullyConnectedLayer(512, ...
    'WeightLearnRateFactor', 10, ...
    'BiasLearnRateFactor', 10, ...
    'Name', 'fc_intermediate1');
dropout1 = dropoutLayer(0.5, 'Name', 'custom_dropout1'); % 40%
dropout

intermediate_fc2 = fullyConnectedLayer(256, ...
    'WeightLearnRateFactor', 15, ...
    'BiasLearnRateFactor', 15, ...
    'Name', 'fc_intermediate2');
dropout2 = dropoutLayer(0.75, 'Name', 'custom_dropout2'); % 50%
dropout

% Final classification layers
final_fc = fullyConnectedLayer(numClasses, ...
    'WeightLearnRateFactor', 20, ...
    'BiasLearnRateFactor', 20, ...
    'Name', 'fc_final');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

% Add these new layers to the graph
lgraph = addLayers(lgraph, intermediate_fc1);

```

```

lgraph = addLayers(lgraph, dropout1);
lgraph = addLayers(lgraph, intermediate_fc2);
lgraph = addLayers(lgraph, dropout2);
lgraph = addLayers(lgraph, final_fc);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the rest of the network
lgraph = connectLayers(lgraph, 'pool5-drop_7x7_s1',
'fc_intermediate1');
lgraph = connectLayers(lgraph, 'fc_intermediate1',
'custom_dropout1');
lgraph = connectLayers(lgraph, 'custom_dropout1',
'fc_intermediate2');
lgraph = connectLayers(lgraph, 'fc_intermediate2',
'custom_dropout2');
lgraph = connectLayers(lgraph, 'custom_dropout2', 'fc_final');
lgraph = connectLayers(lgraph, 'fc_final', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');
% ===== END OF CUSTOM DROPOUT ARCHITECTURE
=====

% Visualize the modified network
figure
plot(lgraph)
title('Modified GoogLeNet with Dropout Layers')

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end

```

```

end
fprintf('Total number of parameters: %d\n', totalParams);

% Original data augmentation settings (kept as in original code)
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
    imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
    imdsValidation);

% Original training options (kept as in original code)
options = trainingOptions('adam', ...
    'MiniBatchSize', 25, ...
    'MaxEpochs', 10, ...
    'InitialLearnRate', 0.00001, ...
    'LearnRateSchedule', 'none', ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Clear workspace and run garbage collection to get reliable initial
memory
clear temp vars;
[~, sysMem1] = memory;
initialTotalMemory = sysMem1.PhysicalMemory.Total -
    sysMem1.PhysicalMemory.Available;
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/
    1024/1024);

```



```

% Start measuring training time
tic;

% Train the model
netTransfer = trainNetwork(augimdsTrain, lgraph, options);

% Calculate and display training time
trainingTime = toc;
fprintf('Total training time: %.2f seconds\n', trainingTime);

% Measure memory after training
[~, sysMem2] = memory;
finalTotalMemory = sysMem2.PhysicalMemory.Total -
sysMem2.PhysicalMemory.Available;
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
fprintf('Total memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

% Evaluate the model on the validation set
tic; % Start timing for inference
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc; % Total inference time
YValidation = imdsValidation.Labels;

% Display inference time metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

% Display random predictions and ground truth
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));
    label2 = YValidation(idx(i));

```

```

    % Fixed the label display order
    label = ["Pred -> ", label1, newline, "Actual -> ", label2];
    title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

```

```

% True positives, false positives, false negatives
TP = sum(actualClassIdxs & predictedClassIdxs);
FP = sum(~actualClassIdxs & predictedClassIdxs);
FN = sum(actualClassIdxs & ~predictedClassIdxs);
TN = sum(~actualClassIdxs & ~predictedClassIdxs);

% Class-wise accuracy (binary accuracy for this specific class)
classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;
totalF1 = totalF1 + f1;

```

end

% Calculate and display macro metrics

macroPrecision = totalPrecision / numel(classes);

macroRecall = totalRecall / numel(classes);

macroF1 = totalF1 / numel(classes);

macroAccuracy = mean(classAccuracies);

% Add F1 Score to the class-wise metrics table

classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);

% Handle NaN values that occur when both precision and recall are
0

classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics

disp('Class-wise Performance Metrics:');

disp(classWiseMetrics);

% Display macro metrics

fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);

fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);

fprintf('Macro Recall: %.2f%%\n', macroRecall*100);

fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics

macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
[macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...

'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics

figure;

uitable('Data', table2cell(macroMetrics), ...

'ColumnName', macroMetrics.Properties.VariableNames, ...

'RowName', [], ...

'Units', 'Normalized', ...

```
'Position', [0.2, 0.2, 0.6, 0.6]);  
title('Macro Performance Metrics');  
  
% Save the trained model and results with a unique name  
save('rice_classification_googlenet_dropout_results.mat',  
'netTransfer', 'accuracy', 'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```

%CODE FOR VGG

```
close all
clear softmaxLayer % Clear any existing softmaxLayer variable
clear classificationLayer % Clear any existing classificationLayer variable
```

```
%unzip('rice.zip');
imds = imageDatastore('Rice_Image_Dataset_Updated',
'IncludeSubfolders', true, 'LabelSource', 'foldernames');
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7,
'randomized');
```

```
% Display a random selection of 16 training images
numTrainImages = numel(imdsTrain.Labels);
idx = randperm(numTrainImages, 16);
figure
for i = 1:16
    subplot(4, 4, i)
    I = readimage(imdsTrain, idx(i));
    imshow(I)
end
```

```
% Load pre-trained VGG19 model
net = vgg19;
```

```
% Analyze the network structure
analyzeNetwork(net);
```

```
% Input size expected by VGG19
inputSize = net.Layers(1).InputSize;
```

```
% Prepare the network for transfer learning
lgraph = layerGraph(net); % Create a layer graph for VGG19
```

```
% Remove the last 3 layers (the fully connected and classification layers)
% For VGG19, these layers are different from GoogLeNet
```

```

lgraph = removeLayers(lgraph, {'fc8', 'prob', 'output'});

% Number of output classes (categories of rice types)
numClasses = numel(categories(imdsTrain.Labels));

% ===== CUSTOM DROPOUT IMPLEMENTATION
=====
% VGG19 already has dropout layers (drop6 and drop7), but we'll
add our custom dropout architecture

% Create intermediate fully connected layers with dropout
intermediateSize1 = 512; % First intermediate layer size
intermediateSize2 = 256; % Second intermediate layer size

% First intermediate layer with dropout
intermediate_fc1 = fullyConnectedLayer(intermediateSize1, ...
    'WeightLearnRateFactor', 10, ...
    'BiasLearnRateFactor', 10, ...
    'Name', 'fc_intermediate1');
dropout1 = dropoutLayer(0.95, 'Name', 'custom_dropout1'); % 40%
dropout

% Second intermediate layer with dropout
intermediate_fc2 = fullyConnectedLayer(intermediateSize2, ...
    'WeightLearnRateFactor', 15, ...
    'BiasLearnRateFactor', 15, ...
    'Name', 'fc_intermediate2');
dropout2 = dropoutLayer(0.95, 'Name', 'custom_dropout2'); % 50%
dropout

% Final classification layers
final_fc = fullyConnectedLayer(numClasses, ...
    'WeightLearnRateFactor', 20, ...
    'BiasLearnRateFactor', 20, ...
    'Name', 'new_fc');
smLayer = softmaxLayer('Name', 'softmax');
classLayer = classificationLayer('Name', 'classoutput');

```

```

% Add these new layers to the graph
lgraph = addLayers(lgraph, intermediate_fc1);
lgraph = addLayers(lgraph, dropout1);
lgraph = addLayers(lgraph, intermediate_fc2);
lgraph = addLayers(lgraph, dropout2);
lgraph = addLayers(lgraph, final_fc);
lgraph = addLayers(lgraph, smLayer);
lgraph = addLayers(lgraph, classLayer);

% Connect the new layers to the rest of the network
lgraph = connectLayers(lgraph, 'drop7', 'fc_intermediate1');
lgraph = connectLayers(lgraph, 'fc_intermediate1',
'custom_dropout1');
lgraph = connectLayers(lgraph, 'custom_dropout1',
'fc_intermediate2');
lgraph = connectLayers(lgraph, 'fc_intermediate2',
'custom_dropout2');
lgraph = connectLayers(lgraph, 'custom_dropout2', 'new_fc');
lgraph = connectLayers(lgraph, 'new_fc', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'classoutput');
% ===== END OF CUSTOM DROPOUT IMPLEMENTATION
=====

% Display the modified network architecture
figure
plot(lgraph)
title('Modified VGG19 with Custom Dropout Layers')

% Calculate total number of parameters in the network
totalParams = 0;
for i = 1:numel(lgraph.Layers)
    if isprop(lgraph.Layers(i), 'Weights') &&
~isempty(lgraph.Layers(i).Weights)
        totalParams = totalParams + numel(lgraph.Layers(i).Weights);
    end
    if isprop(lgraph.Layers(i), 'Bias') &&
~isempty(lgraph.Layers(i).Bias)
        totalParams = totalParams + numel(lgraph.Layers(i).Bias);
    end
end

```



```

    end
end
fprintf('Total number of parameters: %d\n', totalParams);

% Data augmentation settings
pixelRange = [-30 30];
imageAugmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandXTranslation', pixelRange, ...
    'RandYTranslation', pixelRange);

% Create augmented image datastores for training and validation
augimdsTrain = augmentedImageDatastore(inputSize(1:2),
imdsTrain, ...
    'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2),
imdsValidation);

% Training options - same as original
options = trainingOptions('sgdm', ...
    'MiniBatchSize', 25, ...
    'MaxEpochs', 10, ...
    'InitialLearnRate', 0.00001, ...
    'LearnRateSchedule', 'none', ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 1, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Clear workspace and run garbage collection to get reliable initial
memory
clear temp vars;
[~, sysMem1] = memory;
initialTotalMemory = sysMem1.PhysicalMemory.Total -
sysMem1.PhysicalMemory.Available;
fprintf('Initial total memory used: %.2f MB\n', initialTotalMemory/
1024/1024);

```

```

% Start measuring training time
tic;

% Train the model
netTransfer = trainNetwork(augimdsTrain, lgraph, options);

% Calculate and display training time
trainingTime = toc;
fprintf('Total training time: %.2f seconds\n', trainingTime);

% Measure memory after training
[~, sysMem2] = memory;
finalTotalMemory = sysMem2.PhysicalMemory.Total -
sysMem2.PhysicalMemory.Available;
totalMemoryUsed = finalTotalMemory - initialTotalMemory;
fprintf('Total memory used for training: %.2f MB\n',
totalMemoryUsed/1024/1024);

% Evaluate the model on the validation set
tic; % Start timing for inference
[YPred, scores] = classify(netTransfer, augimdsValidation);
inferenceTime = toc; % Total inference time
YValidation = imdsValidation.Labels;

% Display inference time metrics
fprintf('Total inference time: %.2f seconds\n', inferenceTime);
fprintf('Average inference time per image: %.4f seconds\n',
inferenceTime/numel(YValidation));

% Display random predictions and ground truth
idx = randperm(numel(imdsValidation.Files), 4);
figure
for i = 1:4
    subplot(2, 2, i)
    I = readimage(imdsValidation, idx(i));
    imshow(I)
    label1 = YPred(idx(i));

```

```

    label2 = YValidation(idx(i));
    % Fixed the label display order
    label = ["Pred -> ", label1, newline, "Actual -> ", label2];
    title(string(label));
end

% Calculate overall accuracy
accuracy = mean(YPred == YValidation);
fprintf('Overall Validation Accuracy: %.2f%%\n', accuracy*100);

% Get class categories
classes = categories(YValidation);

% Create confusion matrix
figure;
cm = confusionmat(YValidation, YPred);
confusionchart(cm, classes, 'Normalization', 'row-normalized');
title('Confusion Matrix');

% Calculate class-wise metrics
classWiseMetrics = table('Size', [numel(classes), 4], ...
    'VariableTypes', {'string', 'double', 'double', 'double'}, ...
    'VariableNames', {'Class', 'Accuracy', 'Precision', 'Recall'});

classWiseMetrics.Class = classes;

% Calculate macro metrics
totalPrecision = 0;
totalRecall = 0;
totalF1 = 0;

% Initialize array to store class-wise accuracies
classAccuracies = zeros(length(classes), 1);

for i = 1:numel(classes)
    % Get indices for current class
    actualClassIdxs = YValidation == classes{i};
    predictedClassIdxs = YPred == classes{i};

```

```

% True positives, false positives, false negatives
TP = sum(actualClassIdxs & predictedClassIdxs);
FP = sum(~actualClassIdxs & predictedClassIdxs);
FN = sum(actualClassIdxs & ~predictedClassIdxs);
TN = sum(~actualClassIdxs & ~predictedClassIdxs);

% Class-wise accuracy (binary accuracy for this specific class)
classAccuracies(i) = (TP + TN) / (TP + FP + FN + TN);

% Precision and recall
if (TP + FP) > 0
    precision = TP / (TP + FP);
else
    precision = 0;
end

if (TP + FN) > 0
    recall = TP / (TP + FN);
else
    recall = 0;
end

% F1 score
if (precision + recall) > 0
    f1 = 2 * (precision * recall) / (precision + recall);
else
    f1 = 0;
end

% Store in table
classWiseMetrics.Accuracy(i) = classAccuracies(i) * 100;
classWiseMetrics.Precision(i) = precision * 100;
classWiseMetrics.Recall(i) = recall * 100;

% Add to macro totals
totalPrecision = totalPrecision + precision;
totalRecall = totalRecall + recall;

```

```

    totalF1 = totalF1 + f1;
end

% Calculate and display macro metrics
macroPrecision = totalPrecision / numel(classes);
macroRecall = totalRecall / numel(classes);
macroF1 = totalF1 / numel(classes);
macroAccuracy = mean(classAccuracies);

% Add F1 Score to the class-wise metrics table
classWiseMetrics.F1Score = 2 * (classWiseMetrics.Precision .*
classWiseMetrics.Recall) ./ (classWiseMetrics.Precision +
classWiseMetrics.Recall);
% Handle NaN values that occur when both precision and recall are
0
classWiseMetrics.F1Score(isnan(classWiseMetrics.F1Score)) = 0;

% Display class-wise metrics
disp('Class-wise Performance Metrics:');
disp(classWiseMetrics);

% Display macro metrics
fprintf('Macro Accuracy: %.2f%%\n', macroAccuracy*100);
fprintf('Macro Precision: %.2f%%\n', macroPrecision*100);
fprintf('Macro Recall: %.2f%%\n', macroRecall*100);
fprintf('Macro F1 Score: %.2f%%\n', macroF1*100);

% Create summary table for macro metrics
macroMetrics = table({'Accuracy'; 'Precision'; 'Recall'; 'F1 Score'}, ...
    [macroAccuracy*100; macroPrecision*100; macroRecall*100;
macroF1*100], ...
    'VariableNames', {'Metric', 'Value_Percent'});

% Display summary of macro metrics
figure;
uitable('Data', table2cell(macroMetrics), ...
    'ColumnName', macroMetrics.Properties.VariableNames, ...
    'RowName', [], ...

```

```
    'Units', 'Normalized', ...  
    'Position', [0.2, 0.2, 0.6, 0.6]);  
title('Macro Performance Metrics');
```

```
% Save the trained model and results with a unique name  
save('rice_classification_vgg19_dropout_results.mat', 'netTransfer',  
'accuracy', 'trainingTime', ...  
    'totalParams', 'totalMemoryUsed', 'inferenceTime',  
'classWiseMetrics', 'macroMetrics');
```